Файлы. Обработка исключений

Базовые операции

- Открытие / создание файла

  Чтобы получить возможность работать с файлом в Python 3, для начала его необходимо создать. Сделать это можно стандартными средствами операционной системы, перейдя в нужный каталог и создав новый документ с форматом txt. Однако аналогичное действие выполняется и с помощью метода open в языке программирования Python, которому надо передать в качестве параметров название файла и режим его обработки.

  Следующий код демонстрирует получение переменной f ссылки на новый документ. Если запустить эту программу, она создаст текстовый файл test.txt в папке, где хранится исходный код.

```
f = open('text.txt', 'r')
```

  Если же файл с указанным именем test.txt уже существует в каталоге с кодом, программа просто продолжит работу с ним, не создавая новый документ.

  У функции **open** много параметров, нам пока важны 3 аргумента: первый, это имя файла. Путь к файлу может быть относительным или абсолютным. Второй аргумент, это режим, в котором мы будем открывать файл.

Режим	Обозначение
'r'	открытие на чтение (является значением по умолчанию).
'w'	открытие на запись, содержимое файла удаляется, если файла не существует, создается новый.
'x'	открытие на запись, если файла не существует, иначе исключение.
'a'	открытие на дозапись, информация добавляется в конец файла.
'b'	открытие в двоичном режиме.
't'	открытие в текстовом режиме (является значением по умолчанию).
'+'	открытие на чтение и запись

  Режимы могут быть объединены, то есть, к примеру, 'rb' - чтение в двоичном режиме. По умолчанию режим равен 'rt'.

  И последний аргумент, encoding, нужен только в текстовом режиме чтения файла. Этот аргумент задает кодировку.

После выполнения любых манипуляций над файлом, его обязательно следует закрыть с помощью функции close, чтобы гарантированно избежать потери информации.

```
f.close()
```

В предыдущем примере для доступа к файлу был использован относительный путь, который не содержит в себе исчерпывающих сведений о местоположении объекта на жестком диске. Для того, чтобы задать их, необходимо в качестве первого аргумента функции open прописать абсолютный путь. В данном случае документ test.txt будет находиться в корневом каталоге на диске D, а не в папке программы.

```
file = open(r"D:\test.txt", "w")
file.close()
```

Перед строковым литералом мы использовали символ r, для отключения экранирования. Иначе компилятор посчитает последовательность «\t» как символ табуляции и выдаст исключение.

Методы

Объект, который возвращает функция open, содержит ссылку на существующий файл. Также в нем имеется информация о созданном документе, представленная в виде четырех основных полей. Все они описаны в следующей таблице, которая содержит их имена и значение.

Свойство	Значение
name	возвращает имя файла
mode	возвращает режим, в котором был открыт
closed	возвращает true, если файл закрыт и true, если открыт
softspace	возвращает true, если при выводе данных из файла не следует отдельно добавлять символ пробела

Чтобы вывести на экран свойства файла, достаточно воспользоваться оператором доступа, то есть точкой, после чего передать это в качестве параметра уже знакомой функции print.

Например:

```
f = open(r"D:\test.txt", "w")
print(f.name)
f.close()
```

Вывод:

```
D:\test.txt
```

Запись

В Python 3 запись в файл осуществляется с помощью метода write. Метод вызываем у объекта, который ссылается на существующий файл. Важно помнить, что для этого следует предварительно открыть документ с помощью функции open и указать режим записи символом «w». Метод write принимает в качестве аргумента данные, которые нужно поместить в текстовый файл. Следующий пример кода показывает запись строки «hello».

```
file = open("test.txt", "w")
file.write("hello")
file.close()
```

Если необходимо добавить новую информацию к записанным ранее данным, следует заново вызвать функцию open, указав ей в качестве режима работы символ «a». В противном случае все сведения из файла test.txt будут полностью удалены. В приведенном ниже примере кода текстовый документ открывается для дополнительной записи, после чего в него помещается строковый литерал “ world” с пробелом вначале. Таким образом в test.txt будет располагаться «hello world». После всего этого не нужно забывать об обязательном закрытии файла.

```
file = open("test.txt", "a")
file.write(" world")
file.close()
```

Именно так осуществляется самая простая процедура записи данных в текстовый файл. Стоит заметить, что язык программирования Python содержит массу дополнительных средств для более продвинутой работы с документами, которые также включают и улучшенную запись.

Запись бинарных данных

При записи бинарных данных, следует использовать режим «wb». Вот пример записи строки в кодировке utf8:

```
f = open('test.dat', 'wb')
f.write(bytes('строка', 'utf8'))
f.close()
```

Чтение

Для чтения информации из файла в Python 3, следует вызывать метод read через объект, который ссылается на существующий документ. Также необходимо не забывать указывать «r» в качестве второго параметра функции open при открытии текстового файла.

В следующем примере read возвращает информацию из test.txt в метод print, который затем выводит сведения на экран. Как и прежде, программа завершается закрытием документа при помощи метода close. Метод read также может принимать целочисленный параметр, который используется для передачи количества символов для чтения. К примеру, введя 5, программа прочитает только hello.

```
try:
    file = open("test.txt", "r")
    print(file.read())
    file.close()
except FileNotFoundError:
    print('Not found')
except IOError:
    print('Something else')
```

Обратите внимание, что при открытии может возникнуть ошибка. Например, если указанный файл не найден. Поэтому нам пришлось обработать исключение. В Python можно воспользоваться конструкцией with, в таком случае не надо будет обрабатывать исключения и даже закрывать файл. Её рассмотрим ниже.

Есть еще один момент: нельзя делать закрытие в секции finally блока try. Если произойдет исключение при открытии файла, то в секции finally будет ошибка.

Чтение бинарных данных

В случае, если данные бинарного вида — следует использовать «rb» в функции open.

```
try:
    f = open("test.dat", "rb")
    b = f.read(1)
    str = ""
    while True:
        b = f.read(1)
        if b == b'':
            break
        str += b.hex()
    print(str)
    f.close()
except IOError:
    print('error')
```

81d182d180d0bed0bad0b0

Здесь побайтно читается файл. Каждый байт приводит к строковому виду в шестнадцатеричном представлении. С помощью функции print выводится результирующая строка.

with as

Чтобы немного автоматизировать обработку текстовых файлов, рекомендуется использовать связку операторов with as. Благодаря им пропадает необходимость в вызове метода close для документа, который необходимо закрыть, поскольку это происходит автоматически. Все это демонстрируется в следующем фрагменте кода, где происходит считывание данных из test.txt. Как обычно, метод print используется для вывода строковой информации на экран.

```
with open('test.txt', 'r') as file:
    print(file.read())
```

Более того, в этом случае не нужно обрабатывать исключение. Если файла с указанным именем нету, то строки с вложенным кодом в операторе with выполняться не будут.

С помощью рассмотренных возможностей языка программирования Python, пользователь в состоянии достаточно легко выполнять основные операции по работе с чтением и записью информации в файлы.

try-except

Рассмотрим пример в Python конструкции try-except. Она позволяет обработать исключительную ситуацию без необходимости завершения уже работающей программы:

```
print("Program started")
try:
    print("Opening file...")
    f = open("data.txt")
except:
    print("File not found!")
print("Program finished")

Program started
Opening file...
File not found!
Program finished
```

Как видно из кода выше, блок try содержит опасный код, способный привести к ошибке (отсутствие нужного файла), а блок except включает в себя инструкции, которые следует выполнить в случае возникнувшей проблемы. Теперь если требуемый файл не был найден, программа не станет завершаться, о чем свидетельствует последний вывод функции print.

Блоков except может быть несколько, в зависимости от того, какой тип исключения нужно обработать.

Вложенные блоки и else

Блоки try-except могут быть вложенными для более гибкого управления исключениями. В следующем примере демонстрируется попытка открыть текстовый файл и записать в него некую строку. Для каждой цели используется отдельный блок try.

Также в данном примере используется конструкция else, которая выполняется в случае, если в коде не произошло исключений.

В данном случае — else сработает при успешной операции write. По умолчанию файл открывается на чтение в текстовом режиме. Поэтому при открытии файла будем использовать режим «w». В этом режиме файл открывается на запись. Если файла не было — создается новый, если был — перезаписывается.

```
print("Program started")
try:
    print("Opening file...")
    f = open("data.txt", "w")
    try:
        print("Writing to file...")
        f.write("Hello World!")
    except Exception:
        print("Something gone wrong!")
    else:
        print("Success!")
except FileNotFoundError:
    print("File not found!")
print("Program finished")

Program started
Opening file...
Writing to file...
Success!
Program finished
```

finally

Бывают ситуации, когда некие важные действия необходимо совершить вне зависимости от того, было вызвано исключение или нет. Для этого используется блок finally, содержащий набор инструкций, которые нужно выполнить в любом случае. Следующий пример улучшает работу предыдущей программы, добавляя в нее возможность закрывать текстовый файл:

```
print("Program started")
try:
    print("Opening file...")
    f = open("data.txt", "w")
    try:
        print("Writing to file...")
        f.write("Hello World!")
    except Exception:
        print("Something gone wrong!")
    else:
        print("Success!")
    finally:
```

```
        print("Closing file...")
        f.close()
except FileNotFoundError:
    print("File not found!")
print("Program finished")

Program started
Opening file...
Writing to file...
Success!
Closing file...
Program finished
```

Иногда такой подход к работе с текстовыми файлами может показаться слишком сложным, так как код, который его реализует, выглядит громоздким. Специально для этого существует конструкция with/as, позволяющая автоматизировать некоторые методы, такие как закрытие файла у соответствующего объекта. Таким образом, сокращается длина написанного кода:

```
print("Program started")
try:
    print("Writing to file...")
    with open("data.txt", "w") as f:
        f.write("Hello World!")
except Exception:
    print("Something gone wrong!")
print("Program finished")

Program started
Writing to file...
Program finished
```

Как и во всех предыдущих случаях, программа, которая способна правильно обработать все возникающие типы исключений, всегда завершается естественным образом, без прерываний заданного алгоритма. Это значительно повышает комфорт и безопасность ее использования.